FINAL YEAR PROJECT REPORT

FACULTY OF ENGINEERING

BAŞKENT UNIVERSITY

SMART MARKET ASSISTANT

Buket ÇEVİK 21994713

Başak ÖZEK
22096915

Nimet Damla BOYUNAĞA
22098273

Mert Görgülü
22095899

Department: Computer Engineering
Project Supervisor:Doç. Dr. Emre SÜMER
Course Code and Name:BİL494 Bitirme Projesi II
Project Start Date:2023-2024 Fall Semester Project Duration (Semester):2/2
Report Presentation:2023-2024 Spring Semester

This report was approved by the jury members listed below on /05/2024.

Title	Name	Signiture
Doç. Dr.	Emre SÜMER	
Dr. Öğr. Üyesi	Elmas Burcu Mamak EKİNCİ	
Dr. Öğr. Üyesi	Didem ÖLÇER	
Öğr. Gör. Dr.	Oğul GÖÇMEN	

ACKNOWLEDGEMENTS

We would like to express our sincere gratitude to our supervisor, **Assoc. Prof. Dr. Emre Sümer**, who supported us throughout the challenging 2023–2024 fall and spring semesters and whom we consulted for guidance at every stage of this project. We also sincerely thank **Research Assistant Elifnur Erdemir** for her continuous support and encouragement. Additionally, we would like to thank all our instructors who provided us with valuable assistance and patiently answered all our questions during the project.

ABSTRACT

The main purpose of this project is to make users' grocery shopping more effective and enjoyable by combining the traditional shopping experience with contemporary technologies. To achieve this goal, a smart market assistant has been developed that includes image processing and natural language processing technologies. The smart grocery assistant has the capacity to provide consumers with detailed information about products; This will contribute to users making more informed decisions during the shopping process.

Focusing particularly on healthy eating choices, this project aims to offer consumers the opportunity to more easily choose products that support healthy lifestyles. The smart market assistant paves the way for conscious shopping by providing users with detailed information such as the content of the products, nutritional values, usage recommendations and health-related information.

This project also aims to provide an interactive shopping experience through a mobile application. The mobile application will be designed with a user-friendly interface and will provide consumers with the ability to take a closer look at products and guide them in making choices that fit their healthy eating preferences. This way, users will be able to experience their shopping experience not just as a transaction, but as a conscious and enjoyable interaction.

The project aims to go beyond simply offering consumers the opportunity to purchase products and transform the shopping experience into a guidance platform. Raising awareness among consumers about healthy nutrition and offering a shopping experience integrated with technology are the main focuses of this project.

INDEX

ACKNOWLEDGEMENTS 3
1. INTRODUCTION 7
1.1. Project Description 7
1.2. Purpose of the Project 7
1.3. General System Structure / Overview 8
1.4. Project Constraints 8
1.5. Success Criteria 9
1.6. Methods Used in the Project 9
1.7. Functional Requirements 9
1.8. Non-Functional Requirements 10
1.9. Task Distribution 10
1.10. Information About the Next Sections of the Report 10
2. MACHINE LEARNING 11
2.1. Task Description 11
2.2. Machine Learning Model Used 11
2.2.1. YOLOv5 11
2.2.2. Class List 11
2.3. Machine Learning Model Results 11
2.3.1. Training Loss and Validation Accuracy 12
2.3.2. Confusion Matrix 12
2.3.3. Conclusion 13
3. SERVER 14
3.1. Task Description 14
3.2. Flask Server 14
3.2.1. Image Formatting 14
3.2.2. Sending Prediction to Mobile 14
3.2.3. Sending Recipe Generation to Mobile 15
3.3. Ngrok Connection 15
3.3.1. Security and Encryption 15
3.3.2. Conclusion 15
4. MOBILE APPLICATION 16
4.1. Task Description 16
4.2. Goals 16

4.3. Implemented Features 16
4.3.1. Existing User Login Page 16
4.3.2. New User Registration Page 17
4.3.3. Profile Information Page 18
4.3.4. Health Status Page 19
4.3.5. Calorie Calculation Page 20
4.3.6. Main Menu Page 22
4.3.7. Product Recognition Page 22
4.3.8. Add Recognized Product to Cart Page 23
4.3.9. Add to Cart Page 25
4.3.10. My Cart Page 27
4.3.11. Content Page 28
4.3.12. Recipe Generation Page 29
5. NATURAL LANGUAGE PROCESSING (NLP) 31
5.1. GPT-2 Fine-Tuning Summary 31
5.1.1. Token Learning 32
5.2. Changes Made for Recipe Generation 33
5.3. Finding Ingredients 36
5.3.1. Definition of Finding Ingredients 36
5.3.2. Preparing the DataFrame 37
5.3.3. Finding Compatibility 38
5.3.4. Normalization 39
5.3.5. Ingredient Replacement 40
6. DISCUSSION 42
6.1. Advantages 42
6.2. Disadvantages 42
7. PROBLEMS ENCOUNTERED 42
8. CONCLUSION 43
9. REFERENCES 43

1. INTRODUCTION

This project presents a modern approach to the traditional grocery shopping experience by aiming to change existing shopping habits. The main goal of the project is to make the shopping process more effective and enjoyable by integrating image processing and natural language processing technologies. With this technological background, users can access detailed information about products in the market and more easily choose products that match their healthy eating preferences.

The developed mobile application offers an interactive shopping experience with a user-friendly interface and allows consumers to examine products in more detail. In this way, users can make more conscious choices that fit their healthy eating habits and enjoy the shopping process more.

This innovative approach aims to combine technology with shopping and provide users with a unique shopping experience.

1.1. Project Description

The Smart Market Assistant project is a two-semester graduation project that aims to improve the shopping experience of users. Within the scope of this project, customers can quickly and easily scan the products they purchase using their mobile device camera and access detailed information about these products. This information includes product ingredients, nutritional values, usage suggestions, and health-related details.

One of the key features of the project is a system that supports users in shopping according to their health conditions. For example, if a user has allergies or chronic diseases, the application highlights sensitive issues based on this information. Using natural language processing technology, original recipes are generated according to the ingredients provided by the user, and various suggestions are offered.

The main purpose of the project is to provide users with a personalized shopping experience and guide them toward more conscious and healthy shopping decisions. In this context, the Smart Market Assistant project aims to increase customer satisfaction by integrating technology into shopping processes, promote a healthy lifestyle, and offer an experience beyond traditional shopping.

1.2. Purpose of the Project

The Smart Market Assistant project aims to significantly enrich consumers' grocery shopping experiences. Its main purpose is to provide consumers with the information they need to make more conscious and healthy shopping decisions. The specific goals of the project include identifying products in the market using image processing technologies, providing information about these products through natural language processing, and generating recipes based on users' personal preferences.

This allows consumers to quickly access important information such as nutritional values and helps them make more informed purchasing decisions. In addition, by using natural language processing technology, the project adds the ability to generate original recipes based on ingredients, making food recommendations more personalized. As a result, consumers can examine products more closely and access more information to make choices that match their healthy eating preferences.

1.3. General System Structure / Overview

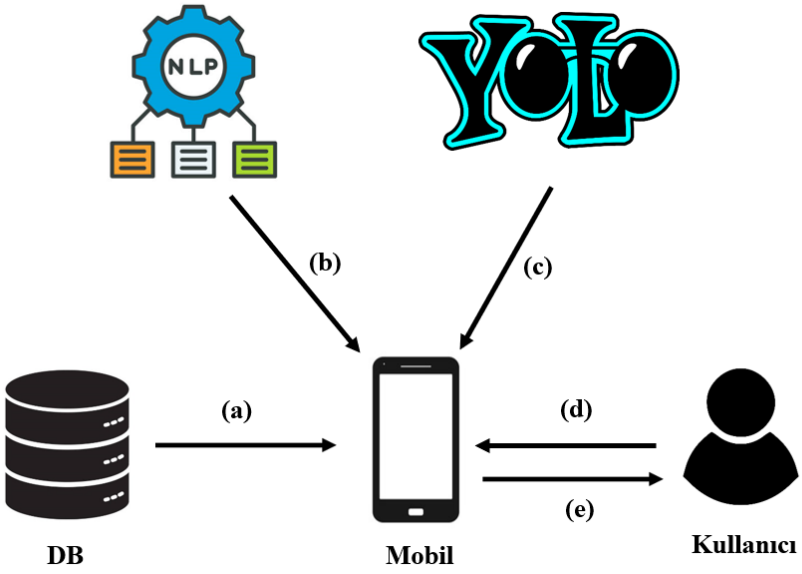

Figure 1.1: General System Architecture

The figure above shows the general architecture of the system.

a. Connection between Database and Mobile:

This connection allows mobile applications to receive data from the database in real time. In this way, mobile users can instantly see updates made in the database. User information is also stored in the database.

b. Connection between NLP (Natural Language Processing) and Mobile

In this part, natural language processing technologies are integrated into the mobile application. This allows the application to understand user language and process commands given in natural language.

c. Connection between YOLO and Mobile:

In this case, a server (usually built using Flask) is set up, and the mobile application connects to this server. The mobile application sends a request to the server, and the server processes the request and sends back the required information, usually via an API. YOLO (You Only Look Once) is an algorithm used for object detection and enables the mobile application to gain object recognition capabilities.

d. and e. Connection between User and Mobile:

These items describe how users interact with mobile devices. This includes how users use the application, enter data, or receive information through the application. This connection is mainly related to user interface and user experience (UI/UX) design.

1.4. Project Constraints

The defined constraints of the project are listed below:

- Adequate lighting must be provided in the market for the camera to work effectively.
- The camera must have sufficient resolution to correctly recognize products.

- The technological infrastructure used may require a stable internet connection inside the market.
- Access to smartphones with compatible operating systems is required for the mobile

1.5. Success Criteria

The success criteria of the project are listed below:

- Image processing algorithms are expected to accurately recognize and classify products in the market.
- The application interface is expected to be easy to navigate and understand for users.
- The application performance is expected to include fast loading times and low latency during use.
- The system is expected to correctly classify products in different categories and provide accurate information about them.
- Recipe suggestions are expected to be generated in a meaningful and appropriate way using natural language processing algorithms.
- application to function.

1.6. Methods Used in the Project

In the project, the Roboflow [1] platform was used to optimize image processing workflows. Roboflow is a platform that simplifies the processes of organizing, labeling, and processing image data. Thanks to this platform, the image processing algorithms used in the project were made more efficient and effective.

In the mobile application part, the user interface was interactively designed using Flutter [2]. Flutter was chosen as a mobile application development framework because it is user-friendly and supports cross-platform compatibility. PyCharm [3] was used to support the backend functionality of the mobile application. This allowed users to easily access the features offered by the smart market assistant and enjoy an interactive shopping experience.

Firebase [4] was used to meet database requirements. Firebase is a cloud-based database and application development platform. With this platform, user data is stored and managed securely, which increases the performance and reliability of the application.

For running Roboflow in the mobile application, Flask [5], PyCharm, and Ngrok [6] were used for the server connection. Flask, a lightweight Python-based web framework, was used to establish the server connection for Roboflow. Ngrok was used to expose the local server to the internet. This technological infrastructure improved the functionality and performance of the mobile application and enabled users to have a better shopping experience.

For natural language processing, tools such as PyCharm and RecipeNLG [7] were used. PyCharm is an integrated development environment used for Python programming, while RecipeNLG is a special library used in natural language processing tasks. These tools helped further personalize the shopping experience and support users in making healthy and informed choices.

This technological infrastructure supports the successful implementation of the project and its goal of providing users with a more conscious and interactive grocery shopping experience.

1.7. Functional Requirements

The functional requirements are listed below:

- The system must provide user registration.
- The system must provide user login.
- The system must allow access to the camera.
- The system must recognize products in images taken by the camera and provide nutritional values.
- The system must be able to identify products containing specific ingredients and suggest recipes related to these ingredients.
- The system must store product information and manage user preferences.

1.8. Non-Functional Requirements

The non-functional requirements are listed below:

- 95% of users should be able to understand the basic functions of the application during their first use.
- Image processing and natural language processing algorithms must work properly.
- User information must be stored securely in the database using encryption.
- The system must allow an increase in the number of users and the addition of new features.
- The data storage used by the system must be sufficient to handle increasing data volume and newly added features throughout the project.

1.9. Task Distribution

The tasks performed by the four students involved in the project are listed below:

- **Buket Çevik:** Mobile application development, database creation, mobile application deployment, poster design.
- **Başak Özek:** Mobile application development, poster design, literature review.
- **Mert Görgülü:** Machine Learning (YOLOv5), server setup.
- **Nimet Damla Boyunağa:** Creation of an NLP-compatible database, integration of the mobile application with GPT-2 fine-tuned RecipeNLG code, literature review.

1.10. Information About the Next Sections of the Report

In the introduction section of the report, the general description and objectives of the project are presented. In the second section, the machine learning stages used for image processing are explained. The third section focuses on server connections. In the fourth section, the mobile application is described in detail. The fifth section focuses on Natural Language Processing (NLP). In the sixth section, a discussion based on the obtained results is presented. The seventh section discusses the problems encountered during the project. Finally, the eighth section presents the conclusion.

2. MACHINE LEARNING

2.1. Task Description

The goal is to develop a module that recognizes and classifies fruits and vegetables in images uploaded by users through the camera or gallery. The scope of the task includes:

1. Processing images received from users' devices.
2. Detecting objects in images and classifying them into predefined classes.
3. Providing feedback to the application about the recognized objects.

The development of this module helps users easily recognize and classify fruits and vegetables and contributes to improving healthy eating and shopping habits.

2.2. Machine Learning Model Used

2.2.1. YOLOv5

In this project, the YOLOv5 (You Only Look Once) model is used to perform fruit and vegetable recognition and classification tasks. YOLOv5 is a modern deep learning model known for its fast and high-accuracy object detection performance.

2.2.2. Class List

The classes used in the model are shown in the tables below.

1- Fruits

Pineapple	Pear	Avocado	Honeydew Melon	Tomato	Galia Melon	Watermelon	Melon
Kiwi	Conference Pear	Red Apple	Lemon	Lime	Banana	Pomegranate	Nectarine
Pink Apple	Orange	Royal Gala Apple	YellowApple	Cherry Tomato	Peach	Green Apple	-

Fruit Classes Table

2- Vegetables

Carrot	Zucchini	Brown Cap Mushroom	Asparagus	Red Pepper	Frying Potato	Cabbage	Eggplant
Leek	Vine Tomato	Yellow Pepper	Garlic –	Cucumber	Onion	Sweet Potato	Orange Pepper
Potato	Green Pepper	-	-	-	-	-	-

Vegetable Classes Table

2.3. Machine Learning Model Results

The results and evaluations obtained during the training of the YOLOv5 model are presented below.

2.3.1. Training Loss and Validation Accuracy

The graphs below show the changes in loss values and validation accuracy during the training process of the model. Figure 2.1 visualizes the model's performance at each epoch.

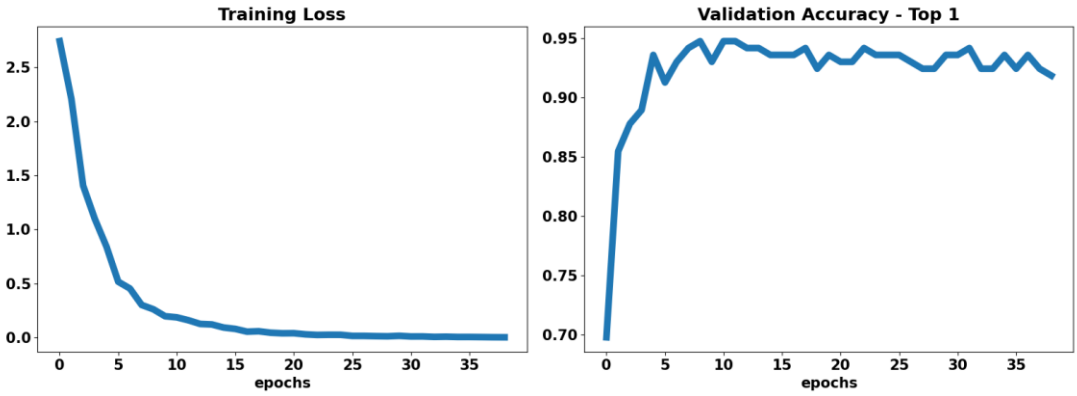

Figure 2.1 Training Loss and Validation Accuracy Graphs

Training Loss:

The decrease in training loss indicates that the model is learning and reducing its error rate.

Validation Accuracy:

The increase in validation accuracy over time shows that the model's generalization ability has improved and the accuracy rate has stabilized.

Best Epoch Selection and Usage:

During the training process, performance metrics obtained across different epochs were analyzed. The epoch that achieved the highest validation accuracy on the validation dataset was selected as the best-performing epoch.

2.3.2. Confusion Matrix

The confusion matrix below shows the classification performance of the model on the test dataset in detail. Each cell represents the relationship between the actual classes and the classes predicted by the model.

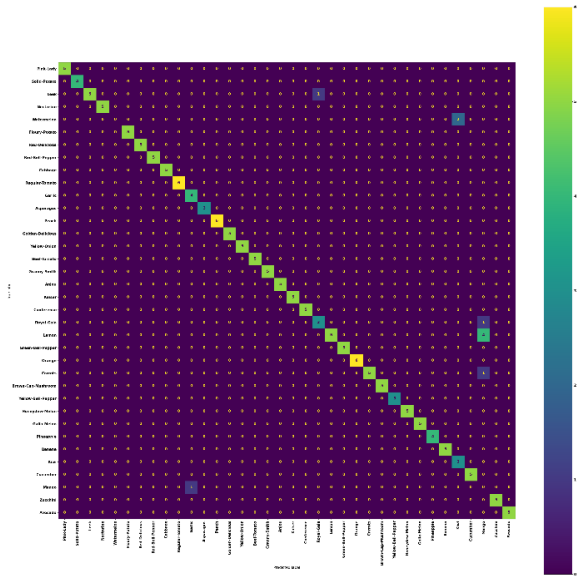

Figure 2.2 Confusion Matrix Graph

Correct Classifications:

High values on the main diagonal (yellow and green) indicate that the model correctly recognizes many classes.

Incorrect Classifications:

Values outside the diagonal indicate that the model experiences confusion between some classes and that there are areas for improvement. For example, the “mango” class can be highlighted. The off-diagonal values show the classes that the model confuses with each other. The model confuses the “lemon” class with the “mango” class, meaning that mango images are sometimes detected as lemons.

2.3.3. Conclusion

The metrics obtained throughout the training process show that the model performs successfully on both the training and validation datasets, with only a few errors. The model achieves a validation accuracy of **94.8%** on the validation dataset.

3. SERVER

3.1. Task Description

This server is an application written using the Flask web framework. The server performs fruit and vegetable recognition on images uploaded by users and sends the results to the mobile application. It also receives input ingredients and allergen information for recipe generation, formats the generated recipe, and sends it back to the mobile application.

The operations performed are listed below:

1. Returning prediction results from the model to the mobile application.
2. Providing feedback about recognized objects to the mobile application in JSON format.
3. Managing the transfer of inputs and outputs between the mobile application and the server for recipe generation.

In this way, a connection between the mobile application and the server is established, and the requirements of the mobile application are handled by the server.

3.2. Flask Server

3.2.1. Image Formatting

The function below decodes an image in base64 format, resizes it, and saves it in JPEG format. The reason the image is received in base64 format is to support data transfer between the mobile application and the server. The image is decoded as JPEG and then sent to the model. Since images may have different sizes, they are resized to a fixed resolution of 1024x978.

```
def base64_to_image(encoded_string, output_path):
    binary_data = base64.b64decode(encoded_string)
    image = Image.open(io.BytesIO(binary_data))
    resized_image = image.resize((1024, 978))  # Resize the image to 1024x978
    resized_image.save(output_path, format='JPEG')  # Save the image as JPEG
```

Figure 3.1: base64_to_image function code snippet

3.2.2. Sending Prediction to Mobile

The function below runs at the /predict endpoint and processes a base64-formatted image sent via a POST request. After decoding and resizing the image, a prediction is made using the model, and the top class is returned to the user in JSON format.

```
@app.route('/predict', methods=['POST'])
...
encoded_image = request.json['image']

base64_to_image(encoded_image, image_path)

result = model.predict(image_path).json()

top_class = result['predictions'][0]['top']
```

```
return jsonify({'prediction': top_class})
```

Figure 3.2: /predict Endpoint

3.2.3. Sending Recipe Generation to Mobile

The endpoint below processes ingredient and allergen data received in a specific data format. It parses the incoming data, calls the run_gen function for recipe generation, and returns the result.

```
@app.route('/tarif_uretimi', methods=['POST'])
def integre():
app.logger.debug(f"Received data: {data}")
...
ingredients_str, allergies_str = data.split(',')
app.logger.debug(f"Ingredients string: {ingredients_str}")
app.logger.debug(f"Allergies string: {allergies_str}")
...
ingredients_list = ingredients_str.split(',')
allergies_list = allergies_str.split(',')
...
tarif = run_gen(ingredients_list, allergies_list)
return tarif
```

Figure 3.3: /tarif_uretimi Endpoint

3.3. Ngrok Connection

Ngrok is a tool that uses the HTTPS (Secure Hyper Text Transfer Protocol) tunneling protocol to create a connection between a local server and a publicly accessible endpoint on the internet. This tunneling allows requests sent to the local server to be accessed over the internet. The image below shows the connection details after installation.

```
ngrok

K8s Gateway API support available now: https://ngrok.com/r/k8sgb

Session Status                online
Account                       Mert Görgülü (Plan: Free)
Version                       3.9.0
Region                        Europe (eu)
Web Interface                 http://127.0.0.1:4040
Forwarding                    https://cbc6-78-175-235-255.ngrok-free.app -> http://localhost:8080

Connections                   ttl     opn     rt1     rt5     p50     p90
                              0       0       0.00    0.00    0.00    0.00
```

Figure 3.4: Ngrok Connection Screen

3.3.1. Security and Encryption

Ngrok uses TLS (Transport Layer Security) encryption to secure its connections. This ensures that data is transmitted safely and securely.

3.3.2. Conclusion

By using Ngrok, the local server was securely exposed to the internet. A connection was established between the mobile application and the server, and smooth data transfer between them was achieved. This method made the development process easier and allowed the application to be tested quickly.

4. MOBILE APPLICATION

4.1. Task Description

This part of the project focuses on creating page designs using code and ensuring interaction with the database. In this section, each page of the user interface is carefully designed and developed in detail. The design process is carried out with the aim of providing a user-friendly experience. In addition, a database connection is established to enable data exchange with the backend of the application. This connection is required to store user activity history, save preferences, and perform other database-related operations. As a result, users can use the application smoothly and easily access the information they need.

4.2. Goals

- Ensuring that the application runs smoothly and quickly
- Proper configuration of the connection between Flutter and Firebase
- Correct operation of the server in the image processing component
- Providing a user-friendly interface
- Continuously optimizing the performance of the application

4.3. Implemented Features

4.3.1. Existing User Login Page

This login screen provides an interface that allows users to sign in to the application. The interface has a user-friendly and visually appealing design. Initially, a visual welcome is presented with a background image, followed by a form for user login.

The form asks users to enter their email and password information. If the required information is missing or entered incorrectly, appropriate error messages are displayed. This helps users provide correct login details and easily fix their mistakes.

When the login button is clicked and the information is correct, the application attempts to log in using the provided email and password. If the login is successful, the user is redirected to the main screen of the application. If the login fails, appropriate error messages are shown.

In addition, when the "Register" button is clicked, the user is redirected to the registration screen, allowing new users to create an account. This improves the user experience by enabling users to easily complete login or registration processes based on their needs.

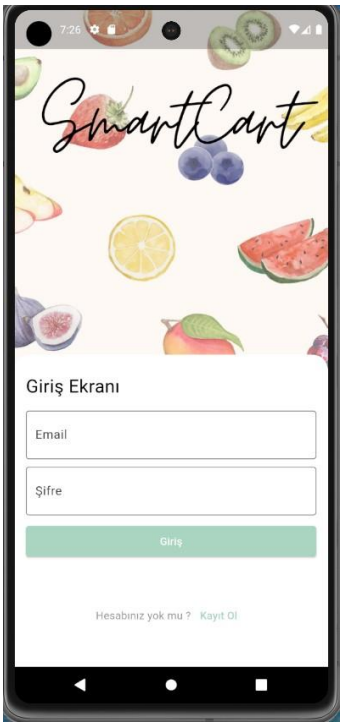

Figure 4.1: User Login Screen

4.3.2. New User Registration Page

This page provides a registration screen that allows users to create a new account. The screen enables users to create an account by entering their email and password information and has a user-friendly design.

In the form area, users are asked to enter their email and password. Warning messages are displayed if these fields are left empty. The email field checks whether the entered email address is valid, and the password field hides the entered password for security.

After entering their information, users can submit the form by clicking the "Start" button. This button verifies the entered information and creates a new account using Firebase Authentication. When the account creation is successful, users are redirected to a screen where they are asked to enter their name and surname. This screen allows users to provide additional information.

At the bottom of the screen, there is a "Login" link for users who already have an account. Clicking this link redirects users to the login screen, allowing them to sign in with their existing accounts.

This registration screen helps users create a new account quickly and smoothly.

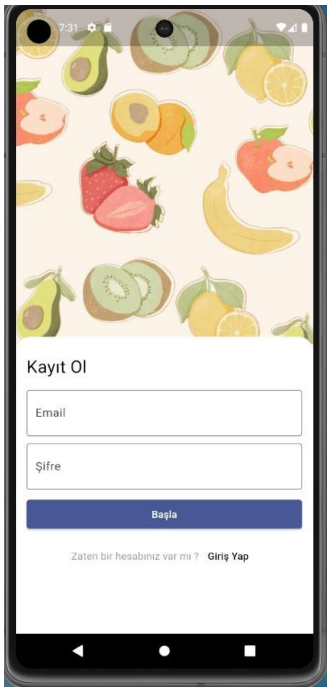

Figure 4.2: User Registration Screen

Additionally, this information is saved in the database so that the user's account details are stored securely. When the user logs in again, this information can be quickly verified. This process helps protect user accounts and ensures secure login. Database integration allows user data to be regularly updated and accessed when needed, enabling users to easily reach their personal settings and history each time they log in.

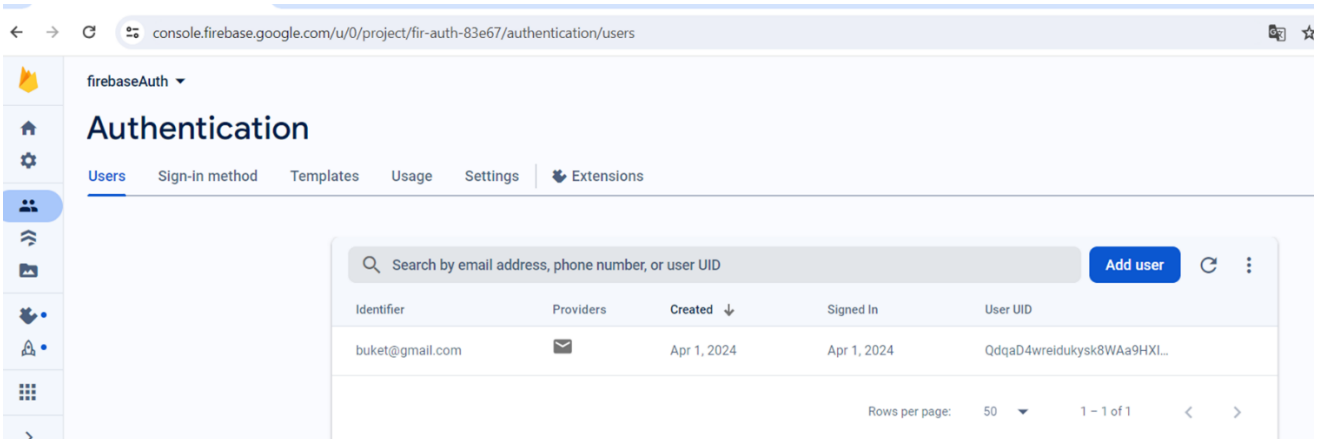

Figure 4.3: Saving User Information to the Database

4.3.3. Profile Information Page

This page allows users to enter additional information during the account creation process. Users enter their first and last names on this screen. The accuracy of the entered information is checked, and users cannot proceed to the next step unless all fields are filled.

At the bottom of the screen, there is a button labeled "Health Status." This button becomes active when all fields are completed and allows the user to move to the next step.

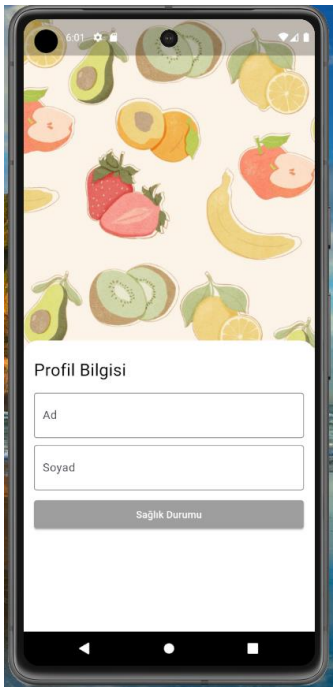

Figure 4.4: Profile Information Screen

4.3.4. Health Status Page

This screen is designed to allow users to select foods they are allergic to. At the top of the screen, there is a search bar that allows users to search among fruits and vegetables.

In the center of the screen, a message saying "Select the foods you are allergic to" is displayed to draw the user's attention. Below this message, options are presented in two categories: fruits and vegetables.

Under each category, specific fruit and vegetable names are listed. Users can select allergenic foods using checkboxes. When users change their selections, the data is automatically saved to the Firestore database.

At the bottom of the screen, there are two buttons. The "Save" button saves the selected foods to the Firestore database and shows a confirmation message. The "Continue" button redirects the user to the next screen called the "Calorie Page."

The main purpose of this screen is to allow users to identify and save their food allergies and then proceed to other diet-related information.

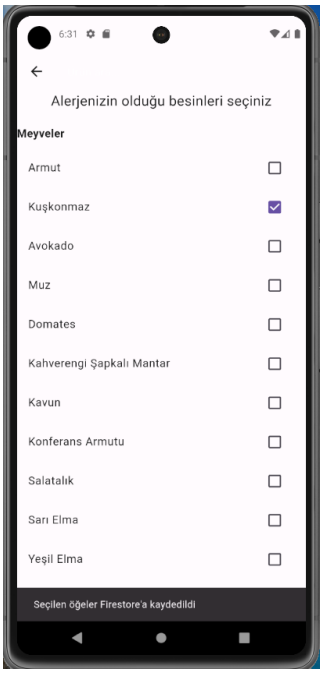

Figure 4.5: Health Status Screen

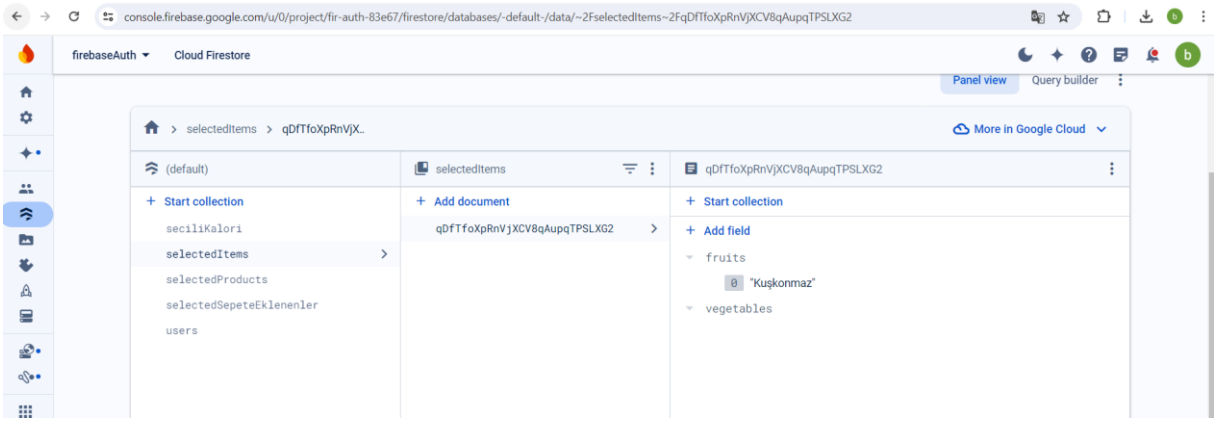

Figure 4.6: Saving User Allergen Information to the Database

4.3.5. Calorie Calculation Page

This screen allows users to calculate their daily calorie needs. In the main section, users are asked to enter their age, height, and weight, which are required for calorie calculation.

Gender and activity level options help calculate calorie needs more accurately based on individual differences. Activity levels include "Sedentary," "Lightly Active," "Moderately Active," and "Active."

Users can also select "Has Diabetes" and "Has Cholesterol" options. These health conditions indicate that carbohydrate, fat, and protein amounts should be adjusted. If selected, the calorie calculation algorithm reduces these values accordingly.

When the user clicks the "Calculate" button, the entered information is checked, and if complete, the daily calorie requirement is calculated. The calculation is based on age, height, weight, gender, and activity level, and also considers diabetes or cholesterol conditions. As a result, daily carbohydrate, fat, and protein amounts are calculated and displayed on the screen.

The calculated values are saved to the user's Firebase Firestore database using the user's UID (Unique User ID). If the user is not logged in, a warning message is shown.

At the bottom of the screen, the calculated carbohydrate, fat, and protein values are displayed as text. There is also a button that redirects the user to the login screen. This allows users to view and save their results and return to the login screen at any time.

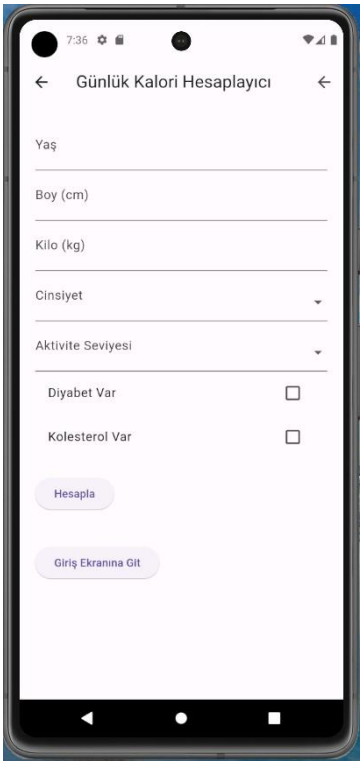

Figure 4.7: Calorie Calculation Screen

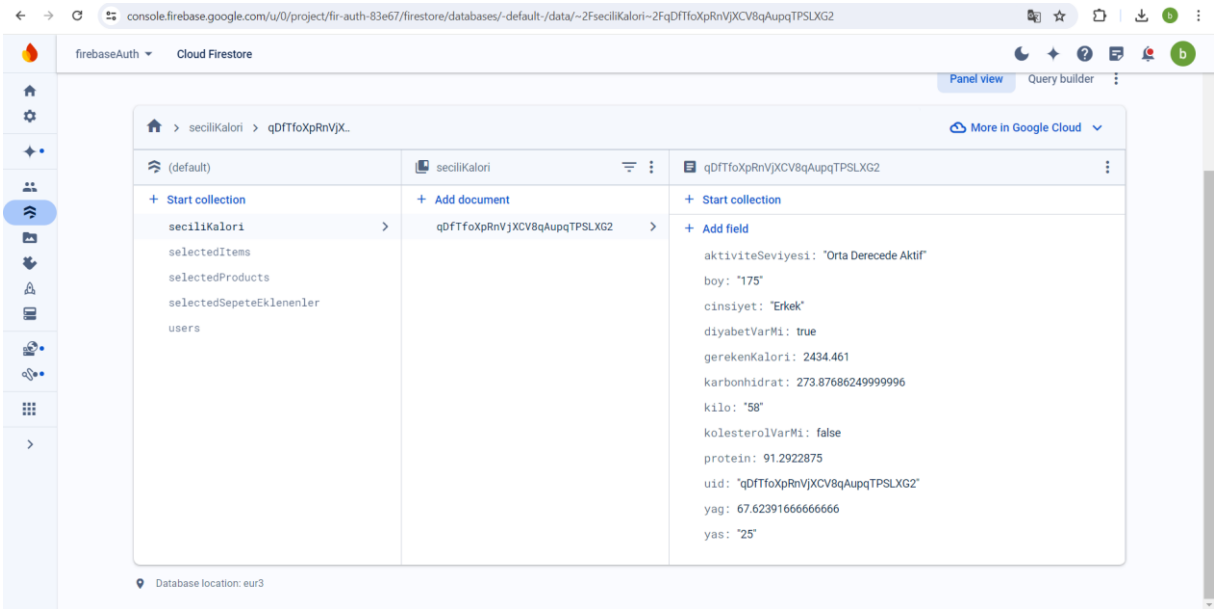

Figure 4.8: Saving Calorie Values to the Database

4.3.6. Main Menu Page

This screen acts as a main menu that allows users to access various features of the application.

In the center of the screen, buttons are displayed to navigate to different features:

- **Recognize Product:** Redirects the user to the product recognition page.
- **Add to Cart:** Takes the user to the page where products can be added to the shopping cart.
- **View Cart:** Allows users to view products in their shopping cart.
- **View Content:** Redirects the user to a page showing detailed product content information.
- **Generate Recipe:** Takes the user to the recipe generation page.

Each button has a navigation function that redirects the user to the relevant page. This screen provides a central control panel that allows users to easily access the main functions of the application.

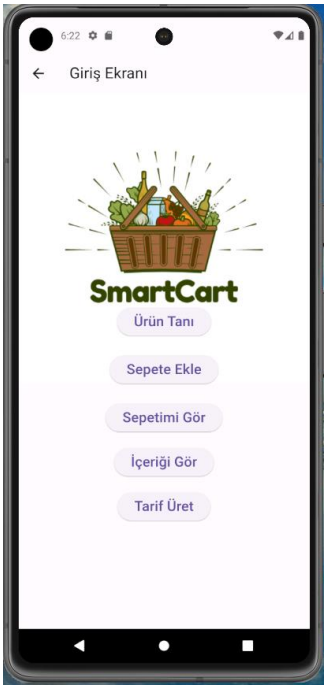

Figure 4.9: Main Menu Screen

4.3.7. Product Recognition Page

This page allows users to select an image from the gallery or camera and send it to an object recognition model via an API to predict the objects in the image.

- **Image Selection:** Users can select an image from the gallery or camera using buttons implemented with the ImagePicker package. The selected image is displayed on the screen.

- **Sending Image to Server:** The selected image is converted to base64 format and sent to the server via an HTTP POST request. The server processes the image and predicts the objects it contains.
- **Receiving and Displaying Prediction:** The prediction received from the server is analyzed and displayed on the screen.
- **Adding to Cart:** When the "Add to Cart" button is clicked, a dialog box appears showing the predicted product name and asks whether the user wants to add it to the cart.

This screen allows users to easily recognize objects in an image and shop conveniently. Thanks to its user-friendly interface, users can quickly recognize products and add them to their cart.

Figure 4.10: Product Recognition Screen

4.3.8. Add Recognized Product to Cart Page

This screen allows users to confirm the accuracy of the predicted product obtained from the previous "Product Recognition" screen and optionally add it to the cart.

The predicted product is obtained through image processing based on an image taken or selected by the user. This prediction is transferred from the previous page and displayed on this screen.

If the user confirms that the predicted product is correct, a checkbox labeled "Add to Cart" is available. When selected, the product is added to the cart and saved to the Firebase Firestore database. If unchecked, the product is removed from the cart and deleted from the database.

This page improves the shopping experience by allowing users to verify predictions and optionally add products to their cart. It also stores user selections in Firebase Firestore, creating data that can be used later.

Figure 4.11: Prediction of the Selected Product

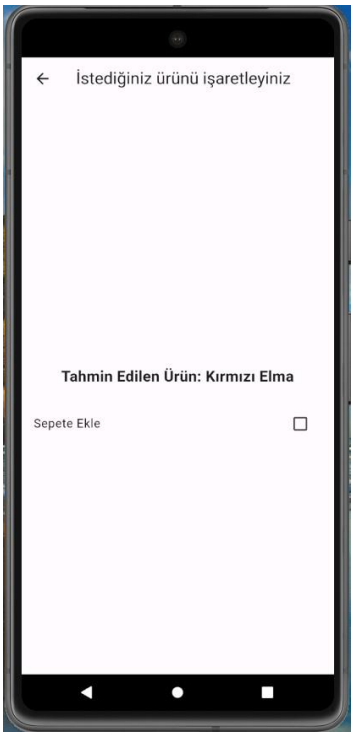

Figure 4.12: Adding Recognized Product to Cart

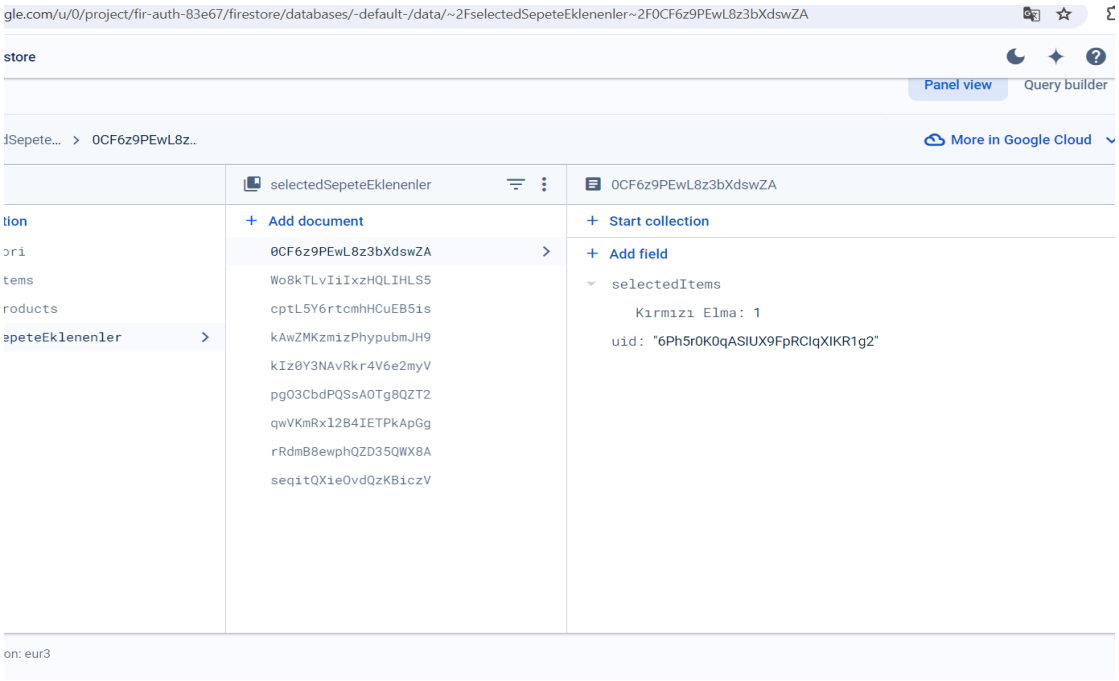

Figure 4.13: Saving Added Product to the Database

4.3.9. Add to Cart Page

This screen allows users to add products to their shopping cart and manage it. A search bar is provided so users can easily find the desired product. Users can add products by searching or selecting from a list. Each added product has increase and decrease buttons to adjust the quantity.

Before adding a product, an allergy warning may be shown. This warning checks the product against the user's previously defined allergen list. If the product is found in the allergy list, a confirmation box warns the user. The user can then choose whether to add the product.

Added products are stored in the user's Firebase Firestore database. A separate document is created for each user, storing the products and quantities in the cart. This ensures that users can always keep their shopping carts up to date.

Users can save their cart by clicking the "Save Cart" button, which sends the cart information to Firebase Firestore and shows a feedback message. Users can also return to the main screen by clicking the "Go to Home Page" button.

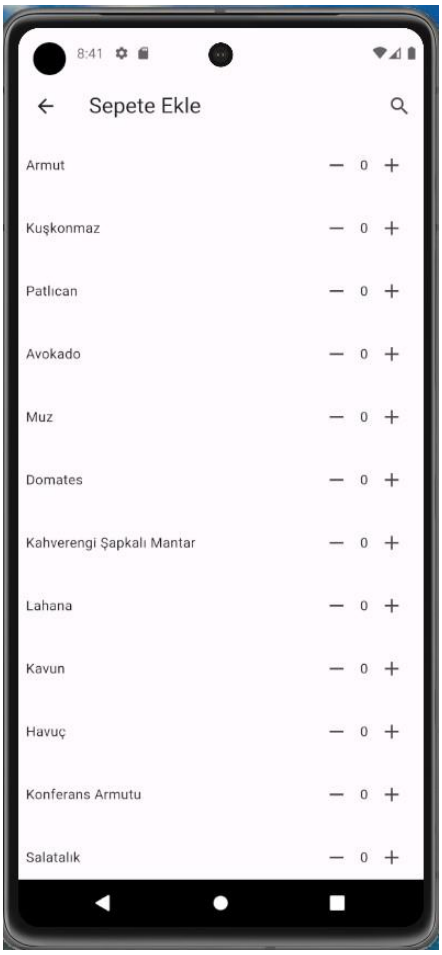

Figure 4.14: Add to Cart Page

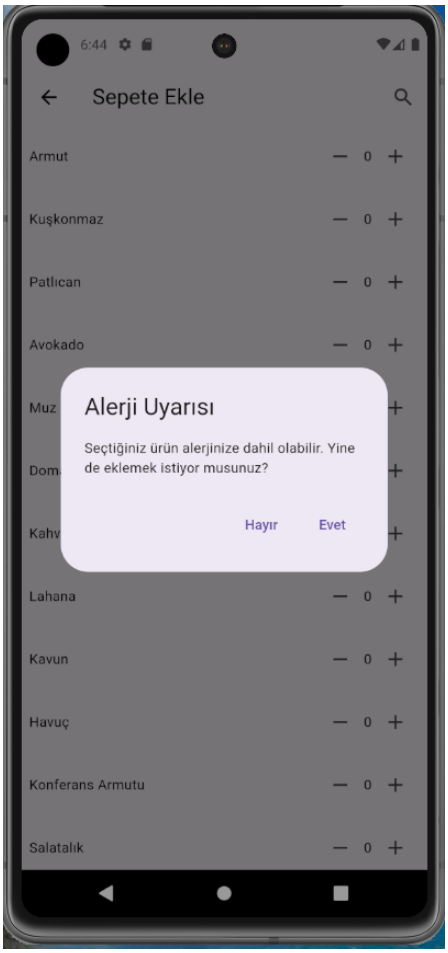

Figure 4.15: Allergy Warning Message

4.3.10. My Cart Page

This screen allows users to view and manage their shopping cart. First, the user's cart data is retrieved from Firebase Firestore. The products and quantities in the cart are stored in a map structure called cartItems.

These items are displayed on the screen using ListView.builder. Each item is placed inside a Dismissible widget, allowing users to remove items by swiping. When an item is removed, it is deleted from both the cart and Firebase Firestore in real time.

If the cart is empty or data has not yet been retrieved from Firestore, a message saying "Your cart is empty." is displayed. If an error occurs, an error message is shown.

This screen allows users to view and manage their cart through Firebase Firestore, making shopping easier and more organized.

Figure 4.16: My Cart Page

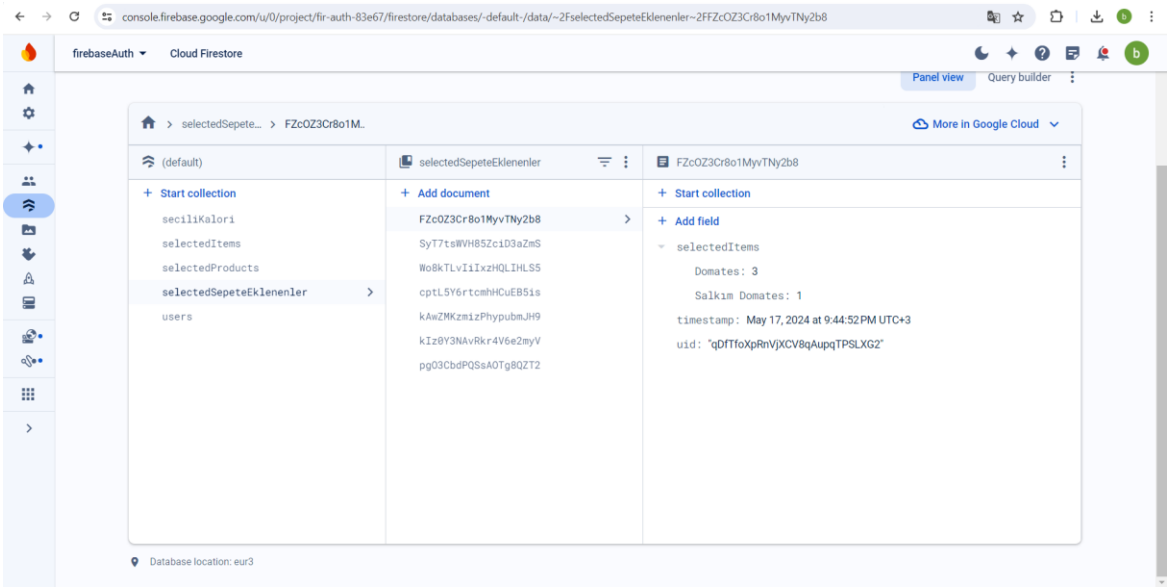

Figure 4.17: Saving My Cart Page to the Database

4.3.11. Content Page

This screen displays the user's daily nutritional information and shopping cart. First, the getIcerikBilgisi method is called using initState to retrieve daily nutritional data. The user's UID is obtained, and daily carbohydrate, protein, and fat values are retrieved from Firebase Firestore. These values are displayed at the top of the screen.

Then, the MyCart component is used to display the list of products in the cart. The cart data is retrieved from Firebase Firestore and displayed using ListView.builder. Each item is placed inside a Dismissible widget, allowing users to remove items from the cart. For each item, total carbohydrate, protein, and fat values are calculated and displayed.

This screen allows users to track their daily nutrition goals and manage their cart easily, helping them maintain a healthy lifestyle.

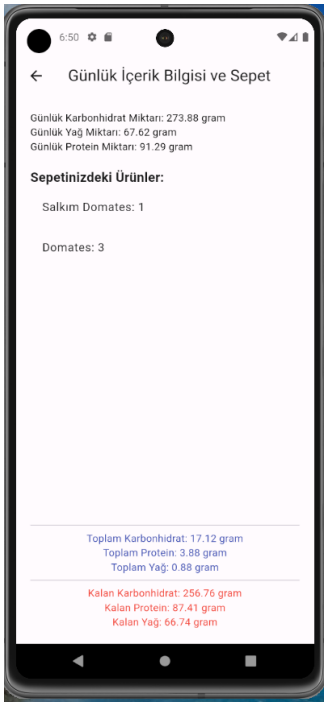

Figure 4.18: Content Page

4.3.12. Recipe Generation Page

This screen allows users to generate custom recipes based on selected ingredients and allergies. The user enters ingredients and allergens, which are sent to a server via an HTTP request, and the server response is processed.

Users first enter ingredients and allergies using TextField components. When the "Send" button is clicked, the entered information is sent to the server.

The sendRequest function sends data to the server using an HTTP POST request. The response is received in JSON format and stored in the _response variable. The _buildResponseDisplay method processes and displays the response on the screen.

If _response contains an error, an error message is displayed. Otherwise, the recipe title, ingredients, and instructions are listed based on the response content.

This allows users to generate custom recipes based on their selected ingredients and allergies and try new meals. This screen personalizes and enriches the cooking experience, making it more enjoyable.

Figure 4.19: Recipe Generation Page

5. NATURAL LANGUAGE PROCESSING (NLP)

Natural Language Processing (NLP) is a field of computer science that focuses on how computers can understand and process the language people speak or write. The main goal of NLP is to analyze texts and speech in natural languages, extract meaningful information from them, and use this information in different applications. This field combines linguistics, computer science, and artificial intelligence.

In this project, a version of the general-purpose language model GPT-2 that is fine-tuned for recipe generation is used.

Fine-tuning is an additional training process that is done after an AI model is trained on a general dataset. The goal is to make the model more suitable for a specific task or dataset. This process improves the model's performance on customized data and helps it respond better to specific needs.

5.1. GPT-2 Fine-Tuning Summary

```
from transformers import (
   AutoModelWithLMHead,
   AutoTokenizer,)

tokenizer = AutoTokenizer.from_pretrained(model_args.tokenizer_name,
cache_dir=model_args.cache_dir)
```

The tokenizer of the GPT-2 model is loaded.

```
model = AutoModelWithLMHead.from_pretrained(
   model_args.model_name_or_path,
   from_tf=bool(".ckpt" in model_args.model_name_or_path),
   config=config,
   cache_dir=model_args.cache_dir,
)
```

The GPT-2 model is loaded.

```
trainer = Trainer(
   model=model,
   args=training_args,
   data_collator=data_collator,
   train_dataset=train_dataset,
   eval_dataset=eval_dataset,

   prediction_loss_only=True,

)
```

The model is trained. A ready fine-tuned model is used.
The trained model files are downloaded from the internet and saved into a folder.

```
special_tokens = {

   "additional_special_tokens": [

      "<TITLE_START>",
      "<TITLE_END>",
      "<INSTR_START>",
      "<NEXT_INSTR>",
      "<INSTR_END>",
      "<INGR_START>",
```

```
        "<NEXT_INGR>",
        "<INGR_END>",
        "<RECIPE_START>",
        "<RECIPE_END>",
        "<INPUT_START>",
        "<INPUT_END>",
        "<NEXT_INPUT>"
    ]
}
```

```
tokenizer.add_special_tokens(special_tokens)

model.resize_token_embeddings(len(tokenizer))
```

In this part, before fine-tuning, some special tokens are added so the model can recognize them. Tokens are basically numeric IDs that represent words, characters, or parts of words.

5.1.1. Token Learning

```
#Extract dictionaries
r_title = recipes.title
r_recipes = recipes.instructions.apply(lambda row: [[value for _, value in item.items()][0] for
item in row])
r_ingredients = recipes.ingredients.apply(lambda row: [[value for _, value in item.items()][0]
for item in row])
```

In this part, the program focuses on three columns in the RecipeNLG dataset:
“title”, “ingredients”, and “recipes” (instructions).
These three columns are taken from the RecipeNLG database. The values of these columns are stored in variables in dictionary format.

```
matches = r_ingredients_lemmas.apply(lambda row: [ingredient for ingredient in
c_ingredients if ingredient in row]) #list lemmas that match the curated dataset

df = "<RECIPE_START> <INPUT_START> " + matches.str.join(" ") + " <INPUT_END>
<INGR_START> " + \
  r_ingredients.str.join(" <NEXT_INGR> ") + " <INGR_END> <INSTR_START> " + \
  r_recipes.str.join(" <NEXT_INSTR> ") + " <INSTR_END> <TITLE_START> " + r_title + "
<TITLE_END> <RECIPE_END>"
```

Here, special tokens are added to the beginning and end of each element in each variable, and the token-added format is placed into a dataframe.

```
trainer = Trainer(
    model=model,
    args=training_args,
    data_collator=data_collator,
    train_dataset=train_dataset,
    eval_dataset=eval_dataset,
    prediction_loss_only=True,
)
```

Training is done on this dataframe.
For example, the model learns to see the <NEXT_INGR> token before each ingredient, so when it sees this token, it knows it should generate an ingredient.

5.2. Changes Made for Recipe Generation

```
def sample_sequence_rec(model, length, context, tokenizer, num_samples=1,
temperature=1, top_k=0, top_p=0.0, device='cpu'):
    end_token = tokenizer.convert_tokens_to_ids(["<RECIPE_END>"])[0]
    context = torch.tensor(context, dtype=torch.long, device=device)
    context = context.unsqueeze(0).repeat(num_samples, 1)
    generated = context
    with torch.no_grad():
        for _ in trange(length):
            inputs = {'input_ids': generated}
            outputs = model(**inputs)  # Note: we could also use 'past' with GPT-2/Transfo-
XL/XLNet (cached hidden-states)
            next_token_logits = outputs[0][0, -1, :] / temperature
            filtered_logits = top_k_top_p_filtering(next_token_logits, top_k=top_k, top_p=top_p)
            next_token = torch.multinomial(F.softmax(filtered_logits, dim=-1), num_samples=1)
            generated = torch.cat((generated, next_token.unsqueeze(0)), dim=1)
            if next_token.item() == end_token:
                break
    return generated
```

End token definition: A stop token is defined. This decides when the model should stop. The context is converted into the required format using the Torch library.

outputs = model(**inputs)

The model is run with input parameters. The model here is the fine-tuned GPT-2 model. Here, the end token is:

<RECIPE_END>

The model keeps generating words until it finds this token (as long as it does not exceed max_length). Since the model is trained to use this token when it finishes generating a recipe, generation stops when this end token appears.

---

def sample_sequence_ing()

The difference of this function from the one above is this line:

end_token = tokenizer.convert_tokens_to_ids(["<INPUT_END>"])[0]

Here, the end token is set as:

<INPUT_END>

The model is trained to use this token when the user finishes entering ingredients or when the model finishes generating ingredients. So when this end token is selected, ingredient generation stops and the model does not continue.

```
parser = argparse.ArgumentParser()
parser.add_argument("--model_type", default="gpt2", type=str,
                    help="Model type selected in the list: " + ", ".join(MODEL_CLASSES.keys()))
parser.add_argument("--model_name_or_path", default="trained", type=str,
                    help="Path to pre-trained model")
parser.add_argument("--prompt", type=str, default="")
parser.add_argument("--length", type=int, default=300)
parser.add_argument("--temperature", type=float, default=1.0)
parser.add_argument("--top_k", type=int, default=0)
parser.add_argument("--top_p", type=float, default=0.9)
parser.add_argument("--no_cuda", action='store_true',
                    help="Avoid using CUDA when available")
parser.add_argument('--seed', type=int, default=42,
                    help="random seed for initialization")
args = parser.parse_args()
```

MAX_LENGTH definition: A maximum length is set to prevent infinite loops during text generation.
Model classes: GPT-2 model and tokenizer classes are defined.
set_seed function: This is used to set the seed value of the random number generator. This helps the model produce repeatable results.
top_k_top_p_filtering function: This function filters the logits (raw model outputs). By using top_k and top_p (nucleus) filtering, the model focuses on the most likely outputs. This improves the quality of generated text and helps produce more meaningful results.
Temperature parameter: A low temperature value (for example 0.7) makes the model generate more consistent and predictable text. A higher temperature value (for example 1.5) makes the model generate more creative and unexpected text [7].

```
if "<INPUT_END>" in prepared_input:
    stage = 1
if stage == 0:
    out = sample_sequence_ing(
    model=model,
    context=context_tokens,
    tokenizer=tokenizer,
    length=args.length,
    temperature=args.temperature,
    top_k=args.top_k,
    top_p=args.top_p,
    device=args.device
    )
    stage = 1
    out = out[0, len(context_tokens):].tolist()
    text_ing = tokenizer.decode(out, clean_up_tokenization_spaces=True)


    focused_ing = raw_text.split(",")
    if focused_ing[-1].endswith(";"):
        focused_ing[-1] = focused_ing[-1][:-1]

    old = text_ing.replace(' <NEXT_INPUT> ',',').replace('<NEXT_INPUT> ',',').replace('
<INPUT_END>',';')
    created_ing = old.split(", ")
    created_ing[-1] = created_ing[-1][:-1]
    created_ing = created_ing[1:]
    excluded_ingredients = set()
    #print(created_ing)
    for i in range(len(created_ing)):
        created_ing[i] = normalize_ingredient(created_ing[i])
    print(created_ing)
    created_ing = focused_ing+created_ing
    final_ing = replace_new(created_ing,excluded_ingredients)


    final_ing[-1] += ";"
    #created_ing[0] += ","
    # Join the modified 'ingredients_list' back into a string with commas

    print("old text_ing",text_ing)
    text_ing = ','.join(final_ing)
    text_ing = text_ing.replace(',', ' <NEXT_INPUT> ').replace(';', ' <INPUT_END>')
```

```
    text_ing = '<NEXT_INPUT> '+text_ing

    print("changed text_ing:",text_ing)
```

In summary: if the <INPUT_END> token exists in the text fed to the model, the model should not generate more ingredients. Then the function with the end token <RECIPE_END> is used, so the system moves to recipe generation.
If the <INPUT_END> token does not exist in the input text, the function below is called:
replace_new(created_ing,excluded_ingredients)

Allergen ingredients are removed from the generated ingredients:

```
last_crt_ing = []
for i in exc_list:
    to_add = normalize_ingredient(i)
    excluded_ingredients.add(to_add)
for i in created_ing:
    if i not in exc_list:
        last_crt_ing.append(i)
```

Calories and gram values are taken from an example Excel file.
If an ingredient name does not exist in the example Excel database (for example, “lactose-free milk”), the program searches for a simpler name (like “milk”) and takes its calorie value.

```
for given_string in given_strings:
    for index, row in df.iterrows():
        *firsts, last = given_string.split()
        if normalize_ingredient(row['Food']) == given_string:
                # If the given string is found, print the values from the 2nd and 3rd columns
                second_column_value = row['Grams']
                third_column_value = row['Calories']
                results_dict[given_string] = {'Grams': second_column_value,
                                              'Calories': third_column_value}

                break  # Exit the loop after finding the first occurrence
        elif normalize_ingredient(row['Food']) == last:
            figured_strings.append(normalize_ingredient(last))

for figured_string in figured_strings:
        for index, row in df.iterrows():
            if normalize_ingredient(row['Food']) == figured_string:
                # If the given string is found, print the values from the 2nd and 3rd columns
                second_column_value = row['Grams']
                third_column_value = row['Calories']
                results_dict[figured_string] = {'Grams': second_column_value,
                                              'Calories': third_column_value}

                break  # Exit the loop after finding the first occurrence
```

The generated recipe is converted into a JSON file using regular expressions. For example:

```
re.search(r'<INPUT_START>(.*?)<INPUT_END>', markdown, re.DOTALL)
if input_match:
    input_ingredients = input_match.group(1).strip()
    input_ingredients_list = input_ingredients.split('<NEXT_INPUT>')
    input_ingredients_list = [item.strip() for item in input_ingredients_list]
```

In the JSON file, the output is separated into 4 lists: title, input ingredients, ingredients (with amounts), and instructions.
The JSON is translated from English to Turkish:

```
translated_recipe_dict = {
```

```
    "title": translate_text(recipe_dict["title"]),
    "input_ingredients": [translate_text(ingredient) for ingredient in
recipe_dict["input_ingredients"]],
    "ingredients": [translate_text(ingredient) for ingredient in recipe_dict["ingredients"]],
    "instructions": [translate_text(instruction) for instruction in recipe_dict["instructions"]]
}

recipe_json = json.dumps(translated_recipe_dict, indent=4, ensure_ascii=False)
```

By setting ensure_ascii = False, Turkish characters are preserved.

Main.py

Ingredients and allergen names received in Turkish from the mobile app are translated into English:

```
ingredients_str = MyMemoryTranslator(target='en-US', source='tr-
TR').translate(ingredients_str)
allergies_str = MyMemoryTranslator(target='en-US', source='tr-TR').translate(allergies_str)

ingredients_list = ingredients_str.split(',')
allergies_list = allergies_str.split(',')

for i in range(len(ingredients_list)):
    ingredients_list[i] = normalize_ingredient(ingredients_list[i])

for i in range(len(allergies_list)):
    allergies_list[i] = normalize_ingredient(allergies_list[i])
```

5.3. Finding Ingredients

5.3.1. Definition of Finding Ingredients

def replace_new(ingredients_list,excluded_ingredients):

This function takes both the ingredients entered by the user and the ingredients generated by the program as parameters.
It asks the user if there is an ingredient they want to change, and the user enters the ingredient to replace.
The function checks whether there is another ingredient that is compatible with the other ingredients in the list. If there is a compatible one, the ingredient is replaced. If not, the ingredient is not changed.

```
Namespace(device=device(type='cpu'), length=300, model_name_or_path='trained', model_type='gpt2', n_gpu=0, no_cuda=False, prompt='
Comma-separated ingredients, semicolon to close the list >>> chicken,milk
  9%|          | 28/300 [00:01<00:18, 15.08it/s]
['black pepper', 'garlic', 'salt', 'onion', 'bell pepper', 'rice', 'pinto bean', 'tomato', 'carrot', 'celery', 'tomato sauce']
Enter the ingredient to replace or type f to proceed with the recipe:garlic
['chicken', 'milk', 'garlic', 'salt', 'onion', 'bell pepper', 'rice', 'pinto bean', 'tomato', 'carrot', 'celery']
Excluded Ingredients: set()
Replacement: No suitable replacement found
Changed List: ['chicken', 'milk', 'garlic', 'salt', 'onion', 'bell pepper', 'rice', 'pinto bean', 'tomato', 'carrot', 'celery']
Enter the ingredient to replace or type f to proceed with the recipe:pinto bean
['chicken', 'milk', 'garlic', 'salt', 'onion', 'bell pepper', 'rice', 'pinto bean', 'tomato', 'carrot', 'celery']
Excluded Ingredients: {'mushroom'}
Replacement: mushroom
Changed List: ['chicken', 'milk', 'garlic', 'salt', 'onion', 'bell pepper', 'rice', 'tomato', 'carrot', 'celery', 'mushroom']
```

Figure 5.1: Ingredient Generation

When “f” is pressed, ingredient generation ends and the system moves to recipe generation. The generated recipe is printed on the screen.

```
Enter the ingredient to replace or type f to proceed
old text_ing <NEXT_INPUT>  black pepper <NEXT_INPUT>
  0%|          | 0/300 [00:00<?, ?it/s]changed text_i
 48%|[redacted]      | 145/300 [00:33<00:35,  4.36it/s]
#   Chicken Casserole   #
 ## Input ingredients ##
`chicken`
`milk`
`chicken`
`milk`
`garlic`
`salt`
`onion`
`bell pepper`
`rice`
`tomato`
`carrot`
`celery`
`mushroom`
```

Figure 5.2: Recipe Generation

```
## Ingredients ##
*   1 tsp.chicken roe
*   1 c.milk
*   2 lb.chicken
*   1/2 c.milk
*   1/2 c.garlic, chopped fine
*   2 tsp.salt
*   1 c.onion, chopped fine
*   1 c.bell pepper, chopped fine
*   1/2 c.rice
*   1 tsp.tomato
*   1/2 c.celery, chopped
*   1/2 c.mushroom
 ## Instructions ##
1)   Blend all.
1)   Brown 2 to 3 minutes in oil.
1)   Put in casserole.
1)   Add vegetables to meat.
1)   Add last 5 ingredients to liquid and cook over low heat for 1 hour.

Comma-separated ingredients, semicolon to close the list >>>
```

Figure 5.3: Recipe Generation

5.3.2. Preparing the DataFrame

At the beginning, the dataset is loaded into the program as a dataframe.

In this part, the dataset is flavor_bible_full.csv. This dataset has two columns: “main” and “pairing”. It shows which ingredients are compatible with the main ingredient.

One limitation of this dataset is that not every ingredient listed in “pairing” also appears as a “main” ingredient. For example, the program may see “chicken” and “garlic” as compatible, but it may not see “garlic” and “chicken” as compatible.

To fix this:

file_path = 'flavor_bible_full.csv'

flavor_bible_df = pd.read_csv(file_path)

Apply the normalization function to the dataset

```
flavor_bible_df['main'] = flavor_bible_df['main'].apply(normalize_ingredient)
flavor_bible_df['pairing'] = flavor_bible_df['pairing'].apply(normalize_ingredient)

# Create a copy of the dataset with 'main' and 'pairing' columns swapped
reversed_pairs_df = flavor_bible_df.rename(columns={'main': 'pairing', 'pairing': 'main'})

# Concatenate the original and reversed pair datasets
augmented_flavor_bible_df = pd.concat([flavor_bible_df, reversed_pairs_df])

# Drop any duplicates that might have arisen from this process
augmented_flavor_bible_df =
augmented_flavor_bible_df.drop_duplicates().reset_index(drop=True)
augmented_flavor_bible_df = augmented_flavor_bible_df.dropna().reset_index(drop=True)
```

A new dataframe is created by swapping “main” and “pairing”. Then it is merged with the original dataframe. Duplicates and rows containing “None” are removed from the final dataframe.

5.3.3. Finding Compatibility

```
# Function to find compatible ingredients
def find_compatible_ingredient(ingredients, compatibility_data):
    normalized_ingredients = [normalize_ingredient(ing) for ing in ingredients]
    filtered_data = compatibility_data[compatibility_data['main'].isin(normalized_ingredients)]
    compatible_ingredients = set(filtered_data['pairing'])
    for ingredient in normalized_ingredients:
        compatible_ingredients = compatible_ingredients.intersection(
            set(filtered_data[filtered_data['main'] == ingredient]['pairing'])
        )
    return compatible_ingredients
```

Filtering compatibility data:

```
filtered_data = compatibility_data[compatibility_data['main'].isin(normalized_ingredients)]
```

This selects only the rows where the “main” ingredient is in the normalized_ingredients list. This limits the process to related ingredients.

Finding compatible ingredients:

```
compatible_ingredients = set(filtered_data['pairing'])
```

This takes all compatible ingredients from the filtered dataset as a set. A set automatically removes duplicates.

Narrowing down by intersection:
This loop finds the compatible ingredients for each ingredient and takes the intersection:

```
for ingredient in normalized_ingredients:
    compatible_ingredients = compatible_ingredients.intersection(
        set(filtered_data[filtered_data['main'] == ingredient]['pairing'])
    )
```

- For each ingredient, it gets the list of ingredients compatible with it.
- Then it intersects this list with the compatible ingredients found so far.
- This ensures only ingredients compatible with *all* entered ingredients remain.

Result:
The function returns a set of ingredients that are compatible with all input ingredients.

5.3.4. Normalization

```
p = inflect.engine()
def normalize_ingredient(ingredient):
    ingredient = ingredient.lower()
    if '(' in ingredient:
        ingredient = ingredient.split('(')[0].strip()
    if ',' in ingredient:
        ingredient = ingredient.split(',')[0].strip()
    if '+' in ingredient:
        ingredient = ingredient.split('+')[0].strip()
    if ':' in ingredient:
        ingredient = ingredient.split(':')[0].strip()
    if ' and ' in ingredient:
        ingredient = ingredient.split('â€')[0].strip()
    if 'â€' in ingredient:
        ingredient = ingredient.split(' and ')[0].strip()
    if '—' in ingredient:
        ingredient = ingredient.split('—')[0].strip()
    if 'cuisine' in ingredient:
        return None  # Mark for removal
    # print("before inf",ingredient)
    try:
        if ingredient[-2:] == "ss":
            pass
        elif p.singular_noun(ingredient):
            ingredient = p.singular_noun(ingredient)
    except:
        pass
    # print(ingredient)
    return ingredient
```

This function is called many times throughout the replace_new function. For example:

```
normalized_ingredients = [normalize_ingredient(ing) for ing in ingredients]
```

It takes an ingredient name as text and removes certain characters from it. In the dataset (flavorbible), some entries like “african cuisine” do not represent a real ingredient, so the function returns None to avoid adding such entries to the dataframe. It also converts ingredient names into singular form.

```
for ing in ingredients_list:
    if ing in ingredients_check.values:
        valid_list.append(ing)
print(valid_list)
replacement, changed_list, excluded_ingredients =
replace_ingredient(ingredient_to_replace, valid_list,
augmented_flavor_bible_df,excluded_ingredients)
```

Here, if GPT-2 generates an ingredient that does not exist in the flavor_bible dataset, compatibility cannot be found for that ingredient. So that ingredient is removed from the list used for compatibility, and compatibility is searched using the remaining ingredients.

5.3.5. Ingredient Replacement

```
def replace_ingredient(ingredient_to_replace, ingredients_list,
compatibility_data,excluded_ingredients):
```

This function performs the following steps:

1) Normalizing ingredients and the ingredient to replace:

```
normalized_ingredients = [normalize_ingredient(ing) for ing in ingredients_list]
normalized_replace = normalize_ingredient(ingredient_to_replace)
```

The inputs are normalized to make ingredient names consistent.

2) Removing the ingredient to replace from the list:

```
if normalized_replace in normalized_ingredients:
    normalized_ingredients.remove(normalized_replace)
else:
    return "Ingredient to replace not found in the list", ingredients_list, excluded_ingredients
```

If the ingredient is not in the list, the function returns an error message.

3) Finding compatible ingredients:

```
compatible_ingredients = find_compatible_ingredient(normalized_ingredients,
compatibility_data)
```

The function finds ingredients that are compatible with the remaining ingredients.

4) Removing unwanted and already existing ingredients:

```
compatible_ingredients = compatible_ingredients.difference(excluded_ingredients)
compatible_ingredients -= set(normalized_ingredients)
```

Ingredients in excluded_ingredients and ingredients already in the list are removed.

5) Choosing a replacement ingredient:

```
replacement_ingredient = None
for ingredient in compatible_ingredients:
    if ingredient != normalized_replace:
        replacement_ingredient = ingredient
        break
```

A replacement ingredient is chosen from the compatible list.

6) Adding the replacement ingredient to the list:

```
if replacement_ingredient:
    changed_ingredients = normalized_ingredients + [replacement_ingredient]
    excluded_ingredients.add(replacement_ingredient)
    return replacement_ingredient, changed_ingredients, excluded_ingredients
else:
    return "No suitable replacement found", ingredients_list, excluded_ingredients
```

If a suitable replacement is found, it is added to the ingredient list and also added to the excluded_ingredients set. The function returns the replacement ingredient, the updated ingredient list, and the updated excluded list. If no suitable replacement is found, it returns an error message and the original ingredient list.

6. DISCUSSION

6.1. Advantages

- **Personalized Shopping Experience:** The system offers shopping suggestions based on factors such as the user's health condition, preferences, and allergies, which creates a personalized experience.
- **Encourages Healthy Eating:** The project guides users toward healthy eating choices and encourages them to make more informed shopping decisions.
- **Image Processing and NLP:** By combining image processing and natural language processing, the system provides detailed product information and helps users make conscious choices.
- **Recipe Suggestions:** The ability to suggest original recipes based on the ingredients the user has gives creative and healthy meal options.
- **Interactive Mobile Application:** A user-friendly mobile interface provides an interactive and easy shopping experience.
- **Fast and Accurate Product Recognition:** Thanks to image processing algorithms, the system can recognize products quickly and accurately.

6.2. Disadvantages

- **Lighting and Resolution Requirements:** The need for enough lighting and a high-resolution camera for effective use may not be practical for some users.
- **Internet Connection Requirement:** The mobile application needs an internet connection for data transfer and natural language processing, which may limit access anytime and anywhere.
- **Dependence on Technology:** If users rely completely on technology for healthy shopping decisions, it may reduce the traditional shopping experience.
- **Security Concerns:** It is important to store and process user information securely, so security concerns may arise.
- **Scalability Challenges:** An increasing number of users and adding new features may cause scalability problems in the system.

7. PROBLEMS ENCOUNTERED

During the development of the Smart Market Assistant project, several technical and user-related problems were encountered. In particular, the need to identify products correctly and analyze their content accurately made it harder to apply the technologies.

While connecting to the server, issues such as network errors, access limitations, or the server being offline were encountered.

Performance problems on the server side may cause delays in processing requests or slow responses.

Finally, it may also be challenging for users to adopt the project and for the project to meet user needs. Changing user habits and moving shopping habits to a digital platform can take time. In addition, different marketing and user training strategies may be needed for users to prefer and use the project.

8. CONCLUSION

The Smart Market Assistant project aims to transform the shopping experience and help users make more informed and healthier shopping decisions through a platform supported by modern technologies. The purpose of the project is to redefine traditional shopping and allow users to easily access detailed information about products in the market and make conscious choices based on this information.

The mobile application provides an interactive shopping experience and allows users to access instant information about products while shopping. With image processing technologies, users can easily access detailed information such as product content and nutrition-related details. In addition, by using natural language processing algorithms, the system can offer original recipes based on the ingredients or nutrition preferences provided by the user. This can guide users toward healthier and more balanced eating.

The database used in the project stores user preferences and shopping history and uses this data to provide personalized suggestions. In this way, the shopping experience becomes more effective and efficient, and users can access suitable products more easily.

With future improvements, AI and machine learning technologies can be developed further to better analyze user preferences and habits and provide more personalized suggestions based on this information. As a result, users can adapt more easily to healthy and sustainable lifestyles, and their shopping experience can become more enjoyable.

9. REFERENCES

[1] https://roboflow.com/ (Last Access: 18 May 2024)
[2] https://flutter.dev/ (Last Access: 18 May 2024)
[3] https://www.jetbrains.com/pycharm/ (Last Access: 18 May 2024)
[4] https://firebase.google.com/ (Last Access: 18 May 2024)
[5] https://flask.palletsprojects.com/en/3.0.x/ (Last Access: 18 May 2024)
[6] https://ngrok.com/ (Last Access: 18 May 2024)
[7] https://recipenlg.cs.put.poznan.pl/ (Last Access: 18 May 2024)